SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA1709**

CO1-AT2-Constraint-Based Problem Solving

Register Number	**192424390**
Name	**Venu G**

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively.*(BL3)*

Assessment Tool Weightage: 20%

QUESTIONS:3 **Total Marks:30**

1. A hospital needs to assign 3 doctors (D1, D2, D3) to 3 shifts (Morning, Afternoon, Night) such that:

   Constraints:

   i. D1 cannot work Night shift
   ii. D2 must work before D3 (Morning < Afternoon < Night)
   iii. Only one doctor per shift
   iv. D3 cannot work Morning
   v. All doctors must be assigned exactly one shift

   Apply Backtracking Search to find a valid assignment with all intermediate steps and constraint checks. State the final valid schedule

2. A robot operates in a grid environment (5×5). It must move from Start (S) to Goal (G). Obstacles block certain paths.

   Constraints:

   i. Movement allowed: Up, Down, Left, Right
   ii. Cost of each move = 1
   iii. Cannot pass through obstacles
   iv. Use Manhattan Distance heuristic

   Using above constraints and BFS Search Algorithm Show step-by-step node expansion and priority queue updates and determine the optimal path and cost**.**

3. An autonomous rescue robot is deployed in a disaster-affected building to locate and rescue survivors. The robot operates in a partially observable environment where some paths are blocked, and it must make decisions with limited information.
   The building is represented as a grid of rooms. The robot starts at position S and must reach a survivor located at G.

   Constraints:

   - The robot can move up, down, left, right

- Some cells are blocked (obstacles) and cannot be traversed
- The robot has limited sensing capability (can only observe adjacent cells)
- Each movement has a cost of 1, but entering risky zones has an additional cost of 2
- The robot must minimize total cost while ensuring safe navigation
- The robot must update its knowledge dynamically (online search scenario)

Tasks:
a) Analyze all the given constraints and explain how they affect the robot's decision-making process.
b) Develop a solution approach using an appropriate search strategy (e.g., Uniform Cost Search / Online Search Agent). Clearly explain the steps involved.
c) Demonstrate how the robot applies AI concepts such as:
- Intelligent agents
- Rationality
- Environment type

d) Justify why your chosen approach is the most suitable for solving this problem under the given constraints.

Code of Conduct:

I _Venu G_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Criterion	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Max Marks
Criteria	**Level 4 (Exemplary)**	**Level 3 (Proficient)**	**Level 2 (Developing)**	**Level 1 (Beginning)**	**Max Marks**
Problem Understanding	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly	6
Constraint Analysis	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization	6
Solution Development	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints	7.5

Application of Concepts	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts	6
Justification	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided	4.5

Question 1: Hospital Doctor Shift Assignment — CSP with Backtracking

Problem Formulation (CSP)

- Variables: D1, D2, D3
- Domain (each): {Morning, Afternoon, Night}
- C1: D1 ≠ Night
- C2: D2 occurs before D3 in order Morning < Afternoon < Night
- C3: All-different (one doctor per shift)
- C4: D3 ≠ Morning
- C5: All 3 doctors assigned (auto-satisfied given 3 vars / 3 values, all-diff)

Domain Reduction (Unary Constraints Applied First)

Doctor	Initial Domain	Reduced Domain
D1	{Morning, Afternoon, Night}	{Morning, Afternoon} (Night removed by C1)
D2	{Morning, Afternoon, Night}	{Morning, Afternoon, Night}
D3	{Morning, Afternoon, Night}	{Afternoon, Night} (Morning removed by C4)

Backtracking Search Tree (Variable order: D1 → D2 → D3)

```
Root
|
+-- D1 = Morning
|  |
|  +-- D2 = Afternoon
|  |  +-- D3 = Night   -> C2: Afternoon < Night OK
|  |                      C3: all-diff OK
|  |                      C4: Night != Morning OK
|  |                      => SOLUTION FOUND
|  |
|  +-- D2 = Night
|     +-- D3 = Afternoon -> C2: Night < Afternoon? FAIL (ordering)
|        => BACKTRACK
|
+-- D1 = Afternoon
   |
   +-- D2 = Morning
   |  +-- D3 = Night   -> C2: Morning < Night OK
```

```
|                C3: all-diff OK
|                C4: Night != Morning OK
|                => ALTERNATE SOLUTION FOUND
|
+-- D2 = Night
   +-- D3: only Afternoon left, but taken by D1
      => domain empty -> BACKTRACK
```

Step-by-Step Constraint Checking (First Branch)

Step	Assignment	Constraint Check	Result
1	D1 = Morning	C1: Morning ≠ Night	Valid
2	D2 = Afternoon	C3: Afternoon ≠ Morning	Valid
3	D3 = Night	C4: Night ≠ Morning; C3: distinct from others; C2: Afternoon < Night	All constraints satisfied

Final Valid Schedule

Doctor	Shift
D1	Morning
D2	Afternoon
D3	Night

Note: A second valid schedule also exists (D1 = Afternoon, D2 = Morning, D3 = Night), since the constraints do not uniquely force D1's shift. Backtracking returns the first satisfying assignment found based on variable/value ordering.

Question 2: Robot Navigation in a 5×5 Grid

Grid Setup

```
Row0: S . . # .
Row1: . # . # .
Row2: . # . . .
Row3: . # # . .
Row4: . . . . G
```

Coordinates as (row, col). S = Start (0,0), G = Goal (4,4), # = Obstacle. Movement: Up/Down/Left/Right, cost = 1 per move.

Heuristic: h(n) = |row - 4| + |col - 4| (Manhattan distance to G)

Since the task asks to combine BFS-style expansion with a Manhattan heuristic, A* Search is applied (f(n) = g(n) + h(n)) — this expands the least-cost node first via a priority queue, exactly like BFS/UCS, but heuristic-guided for efficiency.

Step-by-Step Node Expansion (Priority Queue ordered by f(n))

Step	Node Expanded	g(n)	h(n)	f(n)	New Frontier Added
1	(0,0) S	0	8	8	(0,1) f=8; (1,0) f=8
2	(0,1)	1	7	8	(0,2) f=8
3	(0,2)	2	6	8	(1,2) f=8
4	(1,2)	3	5	8	(2,2) f=8
5	(2,2)	4	4	8	(2,3) f=8
6	(2,3)	5	3	8	(2,4) f=8; (3,3) f=8
7	(3,3)	6	2	8	(4,3) f=8
8	(4,3)	7	1	8	(4,4) f=8
9	(4,4) G	8	0	8	GOAL REACHED

Priority Queue Trace (Simplified)

```
[S:8]
 -> [(0,1):8, (1,0):8]
```

```
-> [(0,2):8, (1,0):8]
-> [(1,2):8, (1,0):8]
-> [(2,2):8, (1,0):8]
-> [(2,3):8, (1,0):8]
-> [(3,3):8, (2,4):8, (1,0):8]
-> [(4,3):8, (2,4):8, (1,0):8]
-> [(4,4):8, ...]  -> GOAL
```

Optimal Path Found

(0,0) -> (0,1) -> (0,2) -> (1,2) -> (2,2) -> (2,3) -> (3,3) -> (4,3) -> (4,4)

Optimal Cost = 8 moves, matching h(0,0) = |0-4| + |0-4| = 8. This confirms the heuristic is admissible and the path found is optimal — no obstacle detour added extra cost beyond the Manhattan lower bound.

Question 3: Autonomous Rescue Robot in a Disaster-Affected Building

a) Analysis of Constraints and Effect on Decision-Making

Constraint	Effect on Decision-Making
Movement restricted to 4 directions	Limits branching factor to at most 4 per node, simplifying but also constraining path options
Blocked cells (obstacles)	Some transitions are invalid; the robot must dynamically prune actions leading into obstacles
Limited sensing (only adjacent cells observable)	Environment is partially observable — robot cannot pre-compute a full optimal path offline; it must discover obstacles as it moves, forcing online/interleaved planning and execution
Movement cost = 1, risky zone cost = +2	Introduces variable path cost, requiring a cost-sensitive search (like UCS) rather than plain BFS, so the robot minimizes total risk-weighted cost, not just steps
Minimize total cost while ensuring safe navigation	Creates a multi-objective tradeoff (shortest path vs. safest path) — robot must balance efficiency against risk exposure
Dynamic knowledge update (online search)	The robot cannot rely on a static pre-built map; it must revise its internal model continuously as new cells are perceived — characteristic of an online search agent

b) Solution Approach: Online Uniform Cost Search (Online-UCS / LRTA*-style)

A pure offline UCS/A* over a fully known graph is not applicable because the full map is unknown in advance (partial observability). The robot must interleave computation and action.

- Initialize the robot at S with an empty/partial knowledge base of the grid (only S and its neighbors known).
- Sense adjacent cells (up/down/left/right) to identify open, blocked, or risky cells.
- Locally compute cost estimates for reachable neighbor cells: g(n) = accumulated cost so far, +2 for entering a risky zone.
- Select the next move using a frontier ordered by cumulative cost (UCS-style) restricted to currently known/sensed cells.
- Move to the selected cell and update the internal map (mark newly sensed cells, remove now-known-blocked cells).
- Backtrack option: if all neighbors of the current state are worse/blocked (dead-end), use memory of visited costs (LRTA*-style) to backtrack to the next-best previously seen alternative.

- Repeat the sense -> decide -> move -> update cycle until the goal G is reached.
- Goal test: terminate when the robot's current position equals the survivor's location G.

Pseudocode

```
function ONLINE_RESCUE_SEARCH(S, G):
    current <- S
    known_map <- {S}
    cost_so_far <- 0
    visited_costs <- {}

    while current != G:
        neighbors <- SENSE_ADJACENT(current)          // partial observability
        UPDATE_KNOWN_MAP(known_map, neighbors)
        valid_moves <- FILTER(neighbors, not blocked)

        if valid_moves is empty:
            current <- BACKTRACK(visited_costs)       // dead-end handling
        else:
            next <- MIN_COST(valid_moves, cost_function)  // UCS-style choice
            move_cost <- 1 + (2 if next is risky else 0)
            cost_so_far <- cost_so_far + move_cost
            visited_costs[current] <- cost_so_far
            current <- next

    return current, cost_so_far
```

c) Demonstration of AI Concepts

Intelligent Agent

- Percepts: adjacent cell status (open/blocked/risky), survivor signal strength, current position
- Actions: Move Up/Down/Left/Right
- Agent Program: sense -> update internal map -> choose least-cost safe move -> act (a model-based, utility-based agent)

Rationality

The robot's action selection at each step maximizes expected performance — reaching G with minimum total cost while avoiding unnecessary risk exposure — given only the information currently available (bounded rationality due to partial observability). It is rational, not necessarily omniscient, since it cannot guarantee the global optimum without full map knowledge.

Environment Type

Property	Classification	Justification
Observability	Partially observable	Only adjacent cells are sensed
Determinism	Deterministic actions, unknown content initially	Moves succeed as expected but the map is not fully known
Dynamics	Static per decision cycle, knowledge updated dynamically	Online updating of internal model as robot moves
Episodic / Sequential	Sequential	Each move affects future options
Discrete / Continuous	Discrete	Grid-based environment
Single / Multi-agent	Single-agent	One robot, no adversary

d) Justification of Chosen Approach

An online, cost-sensitive search (Online-UCS / LRTA*-style) is most suitable because:

- Partial observability rules out any purely offline algorithm (like standard UCS/A* over a fully known graph) — the robot must plan while sensing.
- Variable move costs (normal vs. risky zones) require a cost-aware strategy rather than plain BFS/DFS, which only counts steps.
- Dynamic knowledge updating is essential in a disaster environment where the map may be incompletely known or where debris/obstacles are discovered only upon approach.
- The approach balances optimality (minimizing total cost) with practicality (real-time decision-making with incomplete information) — exactly the situation a rescue robot faces.